CODEM BOOTCAMP TASK
WEEK – 3
JAVASCRIPT EXERCISES

SUBMITED BY
Vasundhara P S
IT – III Year

GITHUB LINK : https://github.com/vasujoe2005/CodemBootCamp.git

Task 1: User Profile Object Manipulation

Requirements

1. Create an object named userProfile with at least the following properties:
    a. name (string)
    b. age (number)
    c. email (string)
    d. isActive (boolean)
2. Update at least two properties using dot notation.
3. Update at least two properties using bracket notation.
4. Add one new property dynamically using bracket notation.
5. Display the entire updated object in the console.
6. Log each property value individually with clear labels.

PROGRAM :

```
let userProfile={
   name: "Vasundhara P S",
   age: 21,
   email: "2315028@nec.edu.in",
   isActive: true
};
console.log("User Profile Object:");
console.log(userProfile);
userProfile.name ="Vasundhara Joe";
userProfile.age =22;
userProfile["email"]="vasuprabakar1985@gmail.com";
userProfile["isActive"]=false;
userProfile["location"]="Arumuganeri";
console.log("Updated User Profile Object:");
console.log(userProfile);
console.log("Name:", userProfile.name);
console.log("Age:", userProfile.age);
console.log("Email:", userProfile.email);
console.log("Is Active:", userProfile.isActive);
console.log("location:", userProfile.location);
```

OUTPUT :

```
PS D:\6th Sem\CODEM\WEEK 3> node task1.js
User Profile Object:
{
  name: 'Vasundhara P S',
  age: 21,
  email: '2315028@nec.edu.in',
  isActive: true
}
Updated User Profile Object:
{
  name: 'Vasundhara Joe',
  age: 22,
  email: 'vasuprabakar1985@gmail.com',
  isActive: false,
  location: 'Arumuganeri'
}
Name: Vasundhara Joe
Age: 22
Email: vasuprabakar1985@gmail.com
Is Active: false
location: Arumuganeri
PS D:\6th Sem\CODEM\WEEK 3>
```

Task 2: Function Modifying an Object

Requirements

1. Create an object representing a user or product with at least three properties.
2. Write a function that:
    a. Accepts the object as a parameter.
    b. Modifies two or more properties of the object.
3. The function must return a formatted string summarizing the updated object data.
4. Call the function and log the returned summary to the console.
5. Verify that the original object has been updated.

PROGRAM :

```
let user = {
   name: "Vasundhara P S",
   age: 21,
   email: "2315028@nec.edu.in",
   isActive: false
};
console.log("User Profile Object:");
console.log(user);
function updateUser(userObj) {
   userObj.age = userObj.age + 1;
   userObj.isActive = true;
   userObj.email = "vasuprabakar1985@gmail.com";
   return `Updated User Details:
Name: ${userObj.name}
Age: ${userObj.age}
Email: ${userObj.email}
Account Active: ${userObj.isActive}`;
}
let result = updateUser(user);
console.log(result);
console.log("Original Object After Update:", user);
```

OUTPUT :

```
PS D:\6th Sem\CODEM\WEEK 3> node task2.js
User Profile Object:
{
  name: 'Vasundhara P S',
  age: 21,
  email: '2315028@nec.edu.in',
  isActive: false
}
Updated User Details:
Name: Vasundhara P S
Age: 22
Email: vasuprabakar1985@gmail.com
Account Active: true
Original Object After Update: {
  name: 'Vasundhara P S',
  age: 22,
  email: 'vasuprabakar1985@gmail.com',
  isActive: true
}
PS D:\6th Sem\CODEM\WEEK 3>
```

Task 3: Nested Shopping Cart Object

Requirements

1. Create a cart object containing:
    a. A nested object or array for items.
    b. Each item must have name, price, and quantity.
2. Access nested properties correctly to calculate the total cost.
3. Calculate the total price using object values only (no hardcoded numbers).
4. Log the final total price to the console with a clear message.
5. Ensure the solution works if item values change.

PROGRAM :

```
let cart = {
   customerName: "Vasu Joe",
   items: [
      { name: "Shirt", price: 1200, quantity: 2 },
      { name: "Jeans", price: 2000, quantity: 1 },
      { name: "Shoes", price: 3000, quantity: 1 }
   ]
};
function calculateTotal(cartObj) {
   let total = 0;
   cartObj.items.forEach(item => {
      total += item.price * item.quantity;
   });
   return total;
}
let totalPrice = calculateTotal(cart);
console.log(`Total Price of Cart for ${cart.customerName}: ₹${totalPrice}`);
cart.items[0].quantity = 3;
let updatedTotal = calculateTotal(cart);
console.log(`Updated Total Price: ₹${updatedTotal}`);
```

OUTPUT :

```
PS D:\6th Sem\CODEM\WEEK 3> node task3.js
Total Price of Cart for Vasu Joe: ₹7400
Updated Total Price: ₹8600
PS D:\6th Sem\CODEM\WEEK 3>
```

Task 4: Looping Through Student Marks

Requirements

1. Create an object where:
    a. Keys are student names.
    b. Values are marks (numbers).
2. Loop through the object using a suitable method.
3. For each student:
    a. Check if the mark is greater than or equal to the pass mark.
    b. Determine pass or fail.
4. Log each student's name, mark, and pass/fail status.
5. Do not hardcode student names inside the loop.

PROGRAM :

```
let studentMarks = {
   Harini: 75,
   Santhiya: 48,
   Joe: 62,
   Vasu: 35
};
let passMark = 50;
for (let student in studentMarks) {
   let mark = studentMarks[student];
   let status = mark >= passMark ? "Pass" : "Fail";
   console.log(`Student: ${student}, Mark: ${mark}, Status: ${status}`);
}
```

OUTPUT :

```
PS D:\6th Sem\CODEM\WEEK 3> node task4.js
Student: Harini, Mark: 75, Status: Pass
Student: Santhiya, Mark: 48, Status: Fail
Student: Joe, Mark: 62, Status: Pass
Student: Vasu, Mark: 35, Status: Fail
PS D:\6th Sem\CODEM\WEEK 3>
```

Task 5: Object Methods Using this

Requirements

1. Create an object with:
   a. At least two properties.
   b. At least one method.
2. The method must:
   a. Use the this keyword to access object properties.
   b. Update at least one property of the object.
3. Log the updated property values from inside the method.
4. Call the method and confirm that the object's data has changed.
5. Log the final object state to the console.

PROGRAM :

```
let employee = {
   name: "Vasu Joe",
   salary: 500000,
   experience: 2,
   promote: function() {
      this.salary += 5000;
      this.experience += 1;
      console.log(`After Promotion:
Name: ${this.name}
Salary: ₹${this.salary}
Experience: ${this.experience} years`);
   }
};
console.log("Before Promotion:",employee);
employee.promote();
console.log("Final Object State:", employee);
```

OUTPUT :

```
PS D:\6th Sem\CODEM\WEEK 3> node task5.js
Before Promotion: {
  name: 'Vasu Joe',
  salary: 500000,
  experience: 2,
  promote: [Function: promote]
}
After Promotion:
Name: Vasu Joe
Salary: ₹505000
Experience: 3 years
Final Object State: {
  name: 'Vasu Joe',
  salary: 505000,
  experience: 3,
  promote: [Function: promote]
}
PS D:\6th Sem\CODEM\WEEK 3>
```

Task 6: Selecting Elements & Updating Content

Requirements

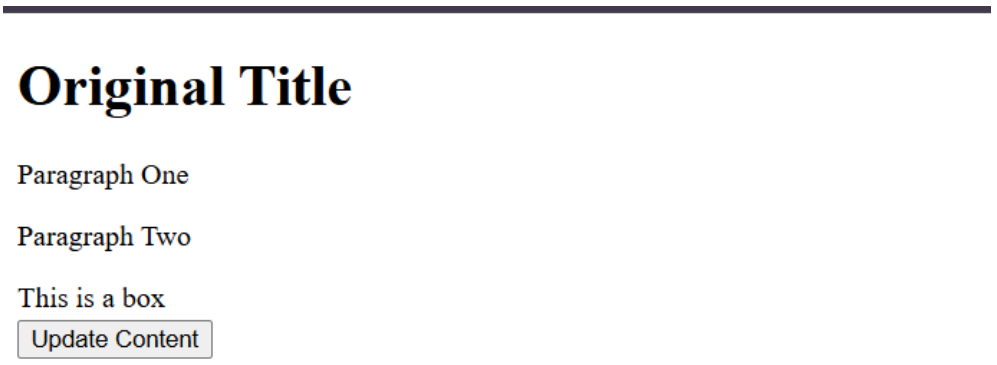
1. Create HTML elements that include:
    a. At least one element with an id
    b. At least two elements sharing the same class
2. Select:
    a. One element using getElementById()
    b. Multiple elements using getElementsByClassName()
    c. One element using querySelector()
3. Update the textContent of each selected element.
4. Change at least two style properties of the selected elements dynamically.
5. Ensure all updates happen using JavaScript only.

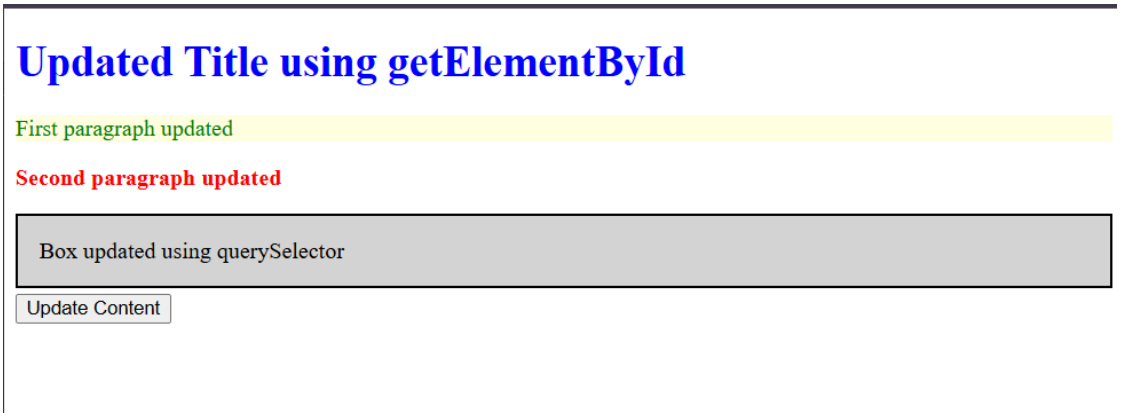
PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Before and After DOM Update</title>
</head>
<body>
  <h1 id="mainTitle">Original Title</h1>
  <p class="info">Paragraph One</p>
  <p class="info">Paragraph Two</p>
  <div class="box">This is a box</div>
  <button onclick="updateContent()">Update Content</button>
  <script>
    function updateContent() {
      let title = document.getElementById("mainTitle");
      let paragraphs = document.getElementsByClassName("info");
      let box = document.querySelector(".box");
      title.textContent = "Updated Title using getElementById";
      paragraphs[0].textContent = "First paragraph updated";
      paragraphs[1].textContent = "Second paragraph updated";
```

```
        box.textContent = "Box updated using querySelector";
        title.style.color = "blue";
        title.style.fontSize = "30px";
        paragraphs[0].style.color = "green";
        paragraphs[0].style.backgroundColor = "lightyellow";
        paragraphs[1].style.color = "red";
        paragraphs[1].style.fontWeight = "bold";
        box.style.border = "2px solid black";
        box.style.padding = "15px";
        box.style.backgroundColor = "lightgray";
    }
  </script>
</body>
</html>
```

OUTPUT :





Task 7: Toggle Class on Multiple Elements

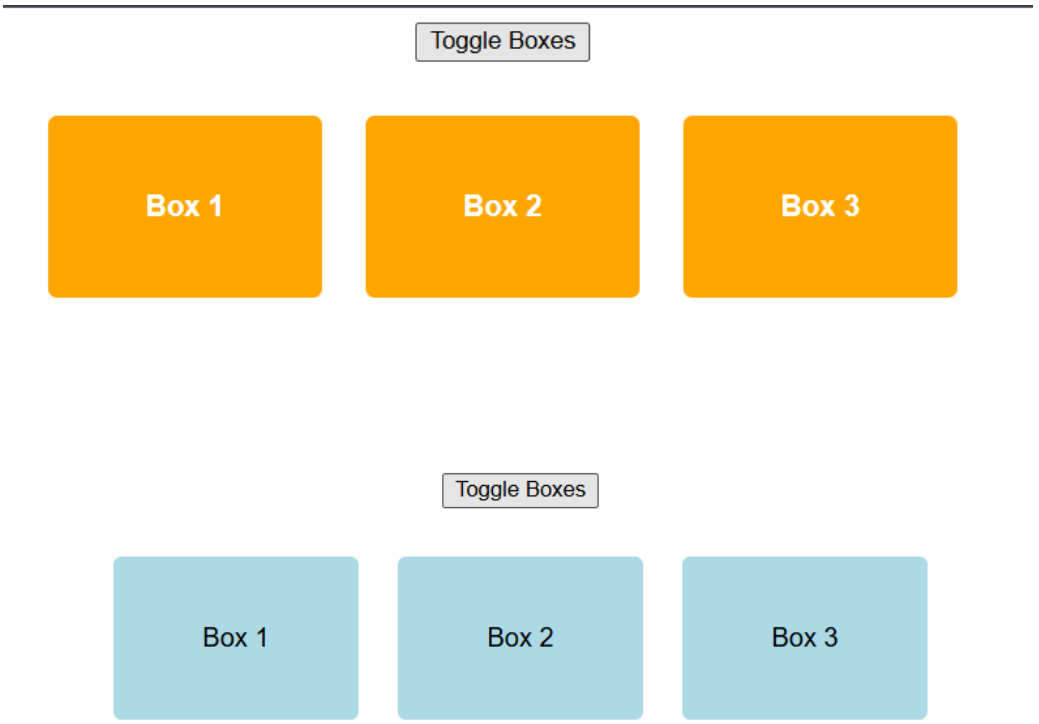
Requirements

1. Create a button that the user can click.
2. Select multiple elements using a class selector.
3. Attach a click event listener to the button.
4. Loop through the selected elements using a loop.
5. Use classList.toggle() to add or remove a CSS class on each element.
6. The toggled class must visibly change the element's appearance.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
   <title>Toggle Boxes</title>
   <style>
      body {
         font-family: Arial;
         text-align: center;
      }
      .container {
         margin-top: 20px;
      }
      .box {
         width: 150px;
         height: 100px;
         margin: 10px;
         display: inline-block;
         background-color: lightblue;
         line-height: 100px;
         border-radius: 5px;
         transition: 0.3s;
      }
      .active {
         background-color: orange;
         color: white;
         font-weight: bold;
      }
   </style>
</head>
<body>
   <button id="toggleBtn">Toggle Boxes</button>
   <div class="container">
      <div class="box">Box 1</div>
      <div class="box">Box 2</div>
      <div class="box">Box 3</div>
   </div>
   <script>
      let button = document.getElementById("toggleBtn");
      let boxes = document.getElementsByClassName("box");
      button.addEventListener("click", function() {
         for (let i = 0; i < boxes.length; i++) {
            boxes[i].classList.toggle("active");
         }
      });
   </script>
</body>
</html>
```

OUTPUT :



Task 8: Dynamically Create a List from Objects

Requirements

1. Create an array containing objects (each object should have at least two properties).
2. Select a container element from the DOM.
3. Dynamically create a list element (ul or ol) using createElement().
4. Loop through the array and:
    a. Create a list item for each object.
    b. Insert object data into the list item using textContent.
5. Append each list item to the list using appendChild().
6. Append the final list to the DOM.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic List from Objects</title>
</head>
<body>
  <div id="listContainer"></div>
  <script>
    let products = [
      { name: "Strawberry", price: 200 },
      { name: "Blueberry", price: 180 },
      { name: "Raspberry", price: 215 }
    ];
    let container = document.getElementById("listContainer");
    let ul = document.createElement("ul");
```

```
        products.forEach(function(product) {
            let li = document.createElement("li");
            li.textContent = `Product: ${product.name}, Price: ₹${product.price}`;
            ul.appendChild(li);
        });
        container.appendChild(ul);
    </script>
</body>
</html>
```

OUTPUT :

- Product: Strawberry, Price: ₹200
- Product: Blueberry, Price: ₹180
- Product: Raspberry, Price: ₹215

Task 9: DOM Traversal

Requirements

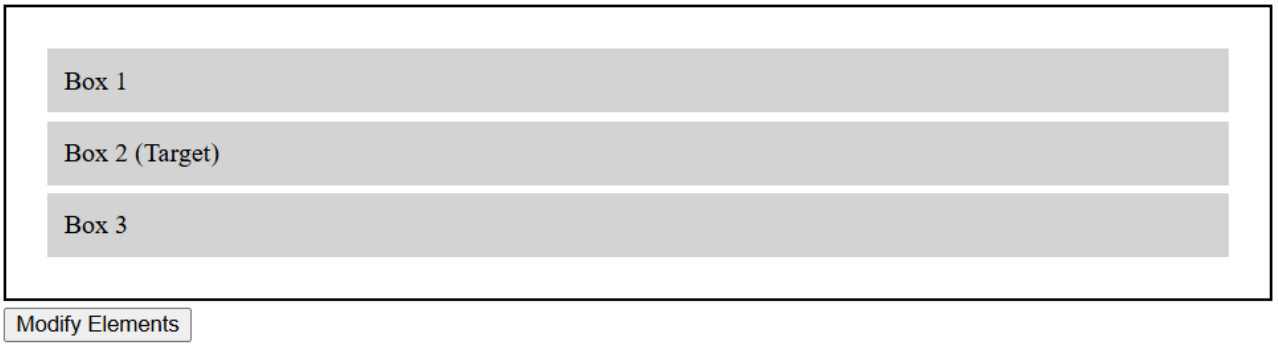
1. Select a specific element from the DOM.
2. Access and modify:
    a. Its parent element
    b. At least one child element
    c. One sibling element
3. Update either the textContent or styles of the traversed elements.
4. Use DOM traversal properties only (no re-selecting elements).
5. Ensure the changes are visible in the browser.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
    <title>DOM Traversal Example</title>
    <style>
        .container {
            padding: 20px;
            border: 2px solid black;
        }
        .box {
            padding: 10px;
            margin: 5px;
            background-color: lightgray;
        }
    </style>
</head>
```

```
<body>
  <div class="container">
    <div class="box">Box 1</div>
    <div class="box" id="target">Box 2 (Target)</div>
    <div class="box">Box 3</div>
  </div>
  <button onclick="traverseDOM()">Modify Elements</button>
  <script>
    function traverseDOM() {
      let target = document.getElementById("target");
      let parent = target.parentElement;
      parent.style.backgroundColor = "lightblue";
      let firstChild = parent.firstElementChild;
      firstChild.textContent = "Box 1 Updated";
      firstChild.style.color = "red";
      let sibling = target.nextElementSibling;
      sibling.style.backgroundColor = "yellow";
      sibling.textContent = "Box 3 Updated";
      target.style.fontWeight = "bold";
      target.textContent = "Target Modified";
    }
  </script>
</body>
</html>
```

OUTPUT :



Task 10: Attribute Manipulation Based on User Interaction

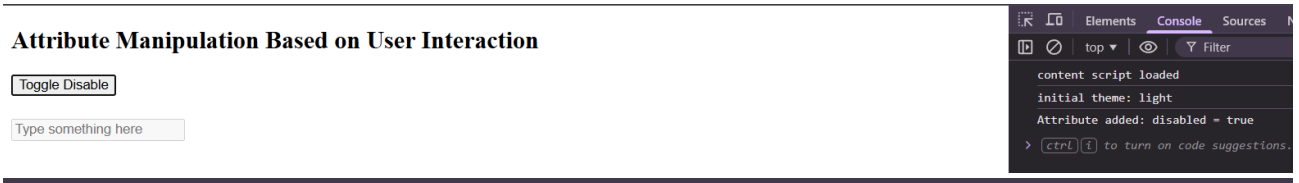
Requirements

1. Create at least one interactive element (button or checkbox).
2. Select a target element whose attribute will be modified.
3. On user interaction:
    a. Check if a specific attribute exists using hasAttribute().
    b. Add the attribute using setAttribute() if it does not exist.
    c. Remove the attribute using removeAttribute() if it exists.
4. The attribute change must affect the element's behavior or appearance.
5. Log the attribute state to the console after each interaction.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Attribute Manipulation</title>
</head>
<body>
  <h2> Attribute Manipulation Based on User Interaction </h2>
  <button id="toggleBtn">Toggle Disable</button>
  <br><br>
  <input type="text" id="textInput" placeholder="Type something here">
  <script>
    const toggleBtn = document.getElementById("toggleBtn");
    const textInput = document.getElementById("textInput");
    toggleBtn.addEventListener("click", function () {
      if (textInput.hasAttribute("disabled")) {
        textInput.removeAttribute("disabled");
        console.log("Attribute removed: disabled = false");
      }
      else {
        textInput.setAttribute("disabled", "true");
        console.log("Attribute added: disabled = true");
      }
    });
  </script>
</body>
</html>
```

OUTPUT :



Task 11: Using the Event Object with Multiple Buttons

Requirements

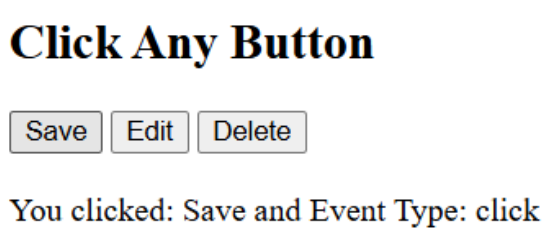
1. Create at least three buttons on the page.
2. Select all buttons using a single selector.
3. Attach the same event listener to each button.
4. Use the event object to:
    a. Identify which button was clicked.
    b. Identify the event type.
5. Display the clicked button's text and event type inside the DOM.
6. Ensure the output updates every time a button is clicked.

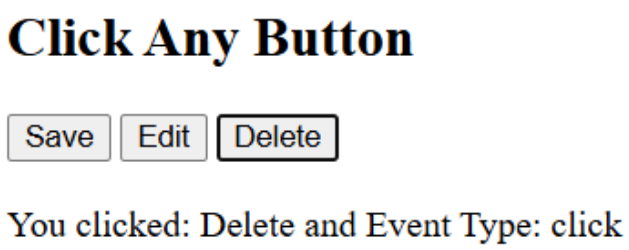
PROGRAM :

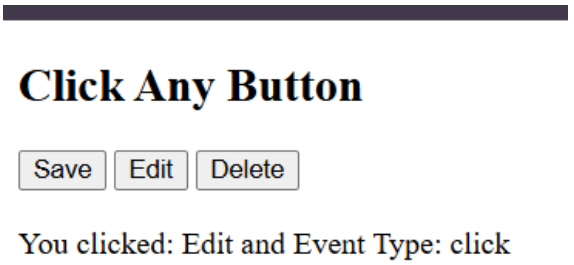
```
<!DOCTYPE html>
<html>
<head>
  <title>Event Object with Multiple Buttons</title>
</head>
<body>

  <h2>Click Any Button</h2>
  <button class="actionBtn">Save</button>
  <button class="actionBtn">Edit</button>
  <button class="actionBtn">Delete</button>
  <br><br>
  <div id="output"></div>
  <script>
    const buttons = document.querySelectorAll(".actionBtn");
    const output = document.getElementById("output");
    buttons.forEach(function (button) {
      button.addEventListener("click", handleClick);
    });
    function handleClick(event) {
      const buttonText = event.target.textContent;
      const eventType = event.type;
      output.textContent = "You clicked: " + buttonText +" and Event Type: " + eventType;
    }
  </script>
</body>
</html>
```

OUTPUT :







Task 12: Form Validation with Prevent Default

Requirements

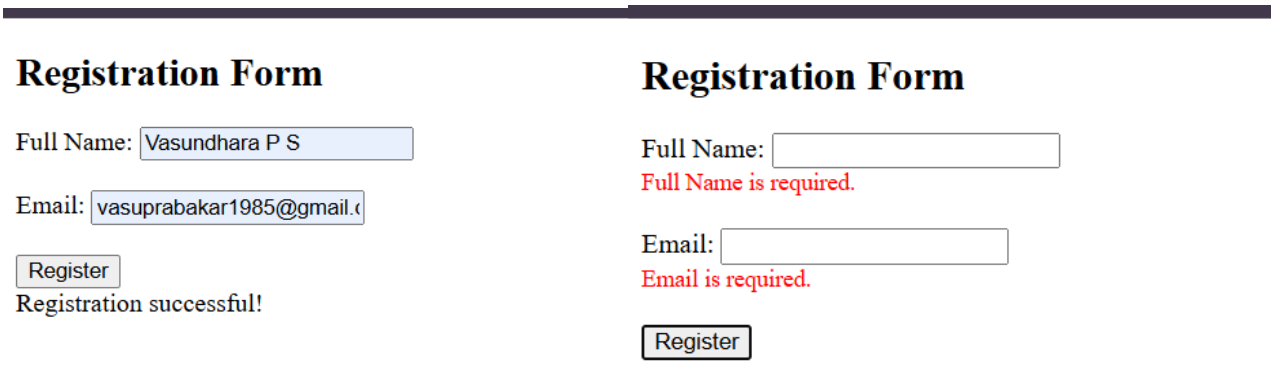
1. Create a form with at least two input fields and a submit button.
2. Attach a submit event listener to the form.
3. Prevent the default form submission behavior.
4. Validate that all required fields are filled.
5. Display an error message inside the DOM for each empty field.
6. Allow form submission logic to continue only when all fields are valid.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Validation</title>
  <style>
    .error {
      color: red;
      font-size: 14px;
    }
  </style>
</head>
<body>
  <h2>Registration Form</h2>
  <form id="myForm">
    <div>
      <label>Full Name:</label>
      <input type="text" id="fullName">
      <div id="nameError" class="error"></div>
    </div>
    <br>
    <div>
      <label>Email:</label>
      <input type="text" id="email">
      <div id="emailError" class="error"></div>
    </div>
    <br>
    <button type="submit">Register</button>
  </form>
  <div id="successMessage"></div>
```

```
  <script>
    const form = document.getElementById("myForm");
    const fullName = document.getElementById("fullName");
    const email = document.getElementById("email");
    const nameError = document.getElementById("nameError");
    const emailError = document.getElementById("emailError");
    const successMessage = document.getElementById("successMessage");
    form.addEventListener("submit", function (event) {
      event.preventDefault();
      let isValid = true;
      nameError.textContent = "";
      emailError.textContent = "";
      successMessage.textContent = "";
      if (fullName.value.trim() === "") {
        nameError.textContent = "Full Name is required.";
        isValid = false;
      }
      if (email.value.trim() === "") {
        emailError.textContent = "Email is required.";
        isValid = false;
      }
      if (isValid) {
        successMessage.textContent = "Registration successful!";
      }
    });
  </script>
</body>
</html>
```

OUTPUT :



Task 13: Real-Time Input Validation

Requirements

1. Create a form with multiple input fields.
2. Validate input values while the user types using an appropriate event.
3. Highlight invalid inputs by:
    a. Adding a CSS class when invalid.
    b. Removing the class when valid.
4. Display real-time validation messages next to each input.
5. Ensure validation feedback updates immediately as input changes.

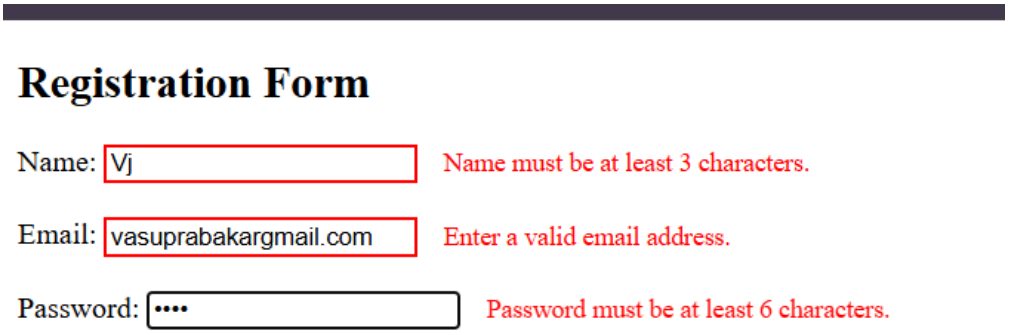
PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
    <title>Real-Time Input Validation</title>
    <style>
        .invalid {
            border: 2px solid red;
        }
        .error-message {
            color: red;
            font-size: 14px;
            margin-left: 10px;
        }
    </style>
</head>
<body>
    <h2>Registration Form</h2>
    <form>
        <div>
            <label>Name:</label>
            <input type="text" id="name">
            <span id="nameError" class="error-message"></span>
        </div>
        <br>
        <div>
            <label>Email:</label>
            <input type="text" id="email">
            <span id="emailError" class="error-message"></span>
        </div>
        <br>
        <div>
            <label>Password:</label>
            <input type="password" id="password">
            <span id="passwordError" class="error-message"></span>
        </div>
    </form>
    <script>
        const nameInput = document.getElementById("name");
        const emailInput = document.getElementById("email");
        const passwordInput = document.getElementById("password");
        const nameError = document.getElementById("nameError");
        const emailError = document.getElementById("emailError");
        const passwordError = document.getElementById("passwordError");
        nameInput.addEventListener("input", function () {
            if (nameInput.value.trim().length < 3) {
                nameInput.classList.add("invalid");
                nameError.textContent = "Name must be at least 3 characters.";
            } else {
                nameInput.classList.remove("invalid");
                nameError.textContent = "";
            }
        });
```

```
    emailInput.addEventListener("input", function () {
      const emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;

      if (!emailPattern.test(emailInput.value)) {
        emailInput.classList.add("invalid");
        emailError.textContent = "Enter a valid email address.";
      } else {
        emailInput.classList.remove("invalid");
        emailError.textContent = "";
      }
    });
    passwordInput.addEventListener("input", function () {
      if (passwordInput.value.length < 6) {
        passwordInput.classList.add("invalid");
        passwordError.textContent = "Password must be at least 6 characters.";
      } else {
        passwordInput.classList.remove("invalid");
        passwordError.textContent = "";
      }
    });
  </script>
</body>
</html>
```

OUTPUT :



Task 14: Keyboard Events Interaction

Requirements

1. Select an element where content will be displayed dynamically.
2. Listen for both keydown and keyup events.
3. Detect specific keys pressed by the user.
4. Update the content or style of the selected element based on the key pressed.
5. Use the event object to identify the key.
6. Ensure the page responds instantly to keyboard input.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Keyboard Events Interaction</title>
</head>
<body>
  <h2>Press Any Key</h2>
  <div id="display">Waiting for key press...</div>
  <script>
    const display = document.getElementById("display");
    document.addEventListener("keydown", function (event) {
      const keyPressed = event.key;
      display.textContent = "Key Down: " + keyPressed;
      if (keyPressed === "Enter") {
        display.textContent = "You pressed ENTER";
      }
      else if (keyPressed === "Escape") {
        display.textContent = "You pressed ESCAPE";
      }
      else if (keyPressed === "a" || keyPressed === "A") {
        display.textContent = "You pressed A";
      }
    });
    document.addEventListener("keyup", function (event) {
      const keyReleased = event.key;
      display.textContent = "Key Up: " + keyReleased;
    });
  </script>
</body>
</html>
```

OUTPUT :

Task 15: Interactive To-Do List with Validation

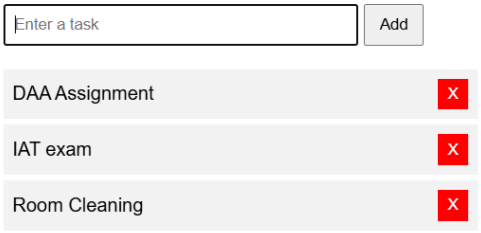
Requirements

1. Create an object named todoData to store tasks added by the user.
2. Dynamically create an input field, an add button, and a task list using JavaScript.
3. Allow users to mark tasks as completed and remove tasks using click events.
4. Update both the DOM and the todoData object when tasks are added or removed.
5. Reuse DOM references and ensure all interactions work on a single page without reloads.

PROGRAM :

```
<!DOCTYPE html>
<html>
<head>
  <title>Interactive To-Do List</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 30px;
    }
    #todoContainer {
      width: 400px;
    }
    input {
      width: 70%;
      padding: 8px;
    }
    button {
      padding: 8px 12px;
      margin-left: 5px;
      cursor: pointer;
    }
    ul {
      list-style: none;
      padding: 0;
      margin-top: 20px;
    }
    li {
      padding: 8px;
      margin-bottom: 5px;
      background: #f2f2f2;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .completed {
      text-decoration: line-through;
      color: gray;
    }
```

```
        .removeBtn {
            background: red;
            color: white;
            border: none;
            padding: 5px 8px;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <script>
        let todoData = {
            tasks: []
        };
        const container = document.createElement("div");
        container.id = "todoContainer";
        const input = document.createElement("input");
        input.type = "text";
        input.placeholder = "Enter a task";
        const addButton = document.createElement("button");
        addButton.textContent = "Add";
        const taskList = document.createElement("ul");
        container.appendChild(input);
        container.appendChild(addButton);
        container.appendChild(taskList);
        document.body.appendChild(container);
        function addTask() {
            const taskText = input.value.trim();
            if (taskText === "") {
                alert("Task cannot be empty");
                return;
            }
            const taskObj = {
                id: Date.now(),
                text: taskText,
                completed: false
            };
            todoData.tasks.push(taskObj);
            renderTask(taskObj);
            input.value = "";
        }
        function renderTask(taskObj) {
            const li = document.createElement("li");
            li.dataset.id = taskObj.id;
            const span = document.createElement("span");
            span.textContent = taskObj.text;
            span.addEventListener("click", function () {
                taskObj.completed = !taskObj.completed;
                span.classList.toggle("completed");
            });
            const removeBtn = document.createElement("button");
            removeBtn.textContent = "X";
            removeBtn.className = "removeBtn";
```

```
        removeBtn.addEventListener("click", function () {
            todoData.tasks = todoData.tasks.filter(
                function (t) {
                    return t.id !== taskObj.id;
                }
            );
            taskList.removeChild(li);
        });
        li.appendChild(span);
        li.appendChild(removeBtn);
        taskList.appendChild(li);
    }
    addButton.addEventListener("click", addTask);
    input.addEventListener("keypress", function (e) {
        if (e.key === "Enter") {
            addTask();
        }
    });
  </script>
</body>
</html>
```

OUTPUT :